Task 2 — WeatherSphere: Advanced Weather Monitoring System

Objective

Build a modern weather web application where users can search for any city and instantly view real-time weather information, climate conditions, environmental data, and a 7-day forecast using a live weather API.

Tools & Technologies

- Python 3
- Flask (Web Framework)
- Requests (API Integration)
- Pandas (Data Handling & CSV Storage)
- JSON Parsing
- HTML5, CSS3, JavaScript
- WeatherAPI.com
- Git & GitHub

How It Works

- The Flask route GET / loads the WeatherSphere dashboard interface.
- Users enter a city name in the search bar and submit the request.
- The application sends a request to the WeatherAPI endpoint using the Requests library.
- The JSON response is parsed to extract weather information such as temperature, humidity, wind speed, pressure, visibility, UV index, sunrise, sunset, and forecast data.
- Search history is automatically stored in a CSV file using Pandas for future analysis.
- Error handling is implemented to display user-friendly messages for invalid city names, API failures, or network issues.
- The weather information is displayed in a responsive dashboard with glassmorphism-inspired UI design and weather cards.

Key Code Snippet

```
def save_search(city, temp):
    data = {
        "Date": [datetime.now()],
        "City": [city],
        "Temperature": [temp]
    }

    df = pd.DataFrame(data)

    if os.path.exists(CSV_FILE):
```

```
        df.to_csv(CSV_FILE, mode="a", header=False, index=False)
    else:
        df.to_csv(CSV_FILE, index=False)


@app.route("/", methods=["GET", "POST"])
def home():

    weather = None

    if request.method == "POST":

        city = request.form.get("city")

        url = f"https://api.weatherapi.com/v1/forecast.json?key={API_KEY}&q=
{city}&days=7&aqi=yes&alerts=yes"

        response = requests.get(url)

        if response.status_code == 200:

            weather = response.json()

            save_search(
                city,
                weather["current"]["temp_c"]
            )

    return render_template(
        "index.html",
        weather=weather
    )
```

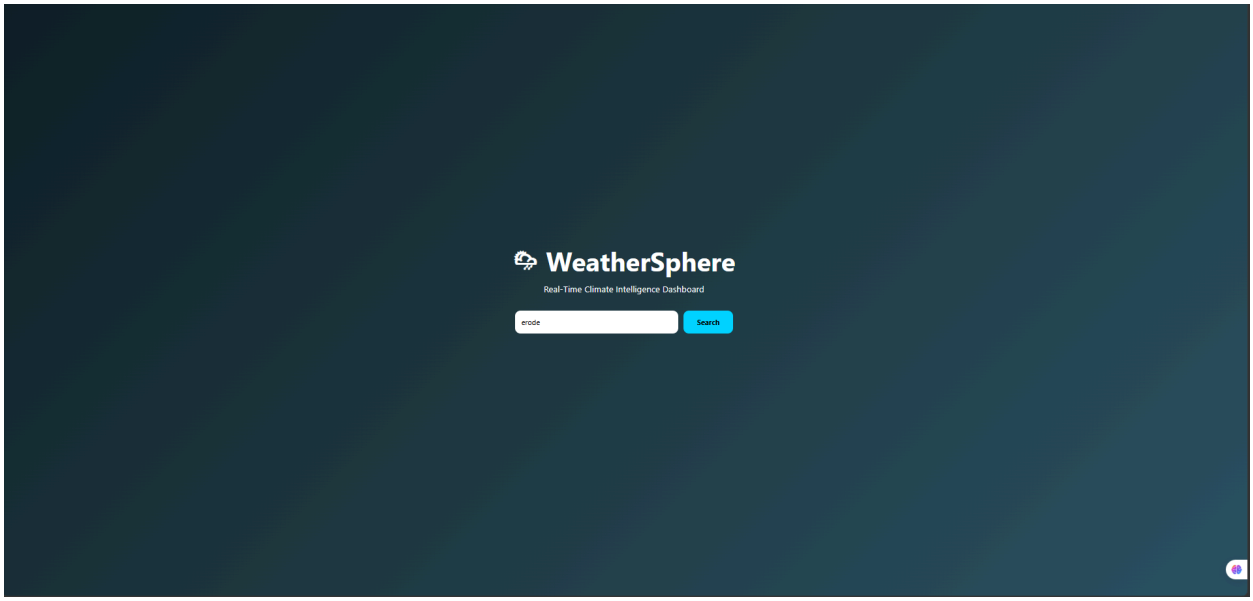
Features Implemented

- Real-Time Weather Search
- Temperature Monitoring
- Humidity Tracking
- Wind Speed Information
- Atmospheric Pressure Display
- Visibility Measurement
- UV Index Monitoring
- Feels-Like Temperature
- Sunrise and Sunset Information
- 7-Day Weather Forecast
- CSV Search History Storage

- Responsive Dashboard UI
- API Error Handling

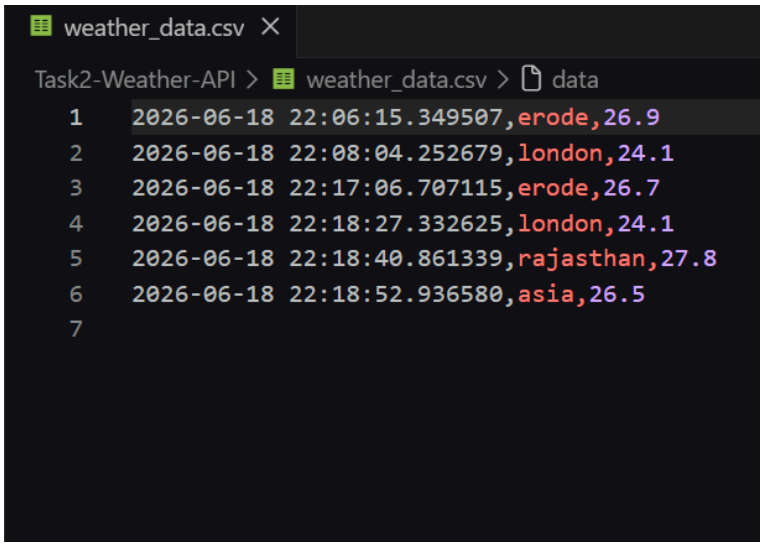
Screenshots

- [WeatherSphere home page before searching }



- [7-Day weather forecast section]

- [CSV weather history file generated]



Submission Links

GitHub Repo
https://github.com/mathiarasan-art01/InternSpark-Internship

Hosted Link
https://weathersphere-wvjt.onrender.com

Notes

A free API key from WeatherAPI.com is required for fetching real-time weather information. The API key should be stored securely and never committed to GitHub repositories. The application uses Flask for backend processing, Requests for API communication, Pandas for CSV data storage, and a responsive HTML/CSS frontend for displaying weather analytics. Complete source code and project files are available in the Task2-WeatherSphere folder of the GitHub repository linked above.